

ServerKit

Dev 1 — SDK Core + Workflow Engine

Your half: `processes/`, `logs/`, `workflows/`, `core/server.py`

Your Role

You are building the heart of ServerKit — the SDK itself. Everything else (the shell, AI layer) depends on what you build here. Dev 2 will plug into the interfaces you define.

Rule: If Dev 2 cannot use your module without looking at your source code, the API is not clean enough. Aim for self-documenting method names and clear return types.

Your Modules

Module	Responsibility
core/server.py	The Server class — entry point for all SDK usage
processes/	ProcessManager, Process object, ProcessFactory
logs/	LogManager, LogFile object
workflows/	Workflow, WorkflowStep, WorkflowManager, StepFactory

Execution Model — Eager

Use eager execution throughout. Each chained method runs immediately, filters `self.data`, and returns `self`. This keeps implementation simple and correct for local data sources.

```
class ProcessCollection:
    def __init__(self, data):
        self.data = data          # list of Process objects

    def memory_above(self, mb):
        self.data = [p for p in self.data if p.memory_mb > mb]
        return self              # return self for chaining

    def sort_by_memory(self):
        self.data = sorted(self.data, key=lambda p: p.memory_mb,
reverse=True)
        return self

    def all(self):
        return self.data         # terminal – returns list
```

Never return raw lists mid-chain. Always return `self`. Only terminal methods like `.all()` and `.summarize()` return a final value.

core/server.py — The Server Class

This is the entry point. Every SDK interaction starts here. Keep it thin — it delegates to managers.

```
from serverkit.processes.manager import ProcessManager
from serverkit.logs.manager import LogManager
from serverkit.workflows.manager import WorkflowManager

class Server:
    def __init__(self):
        self._process_manager = ProcessManager()
        self._log_manager = LogManager()
        self._workflow_manager = WorkflowManager()

    def processes(self) -> ProcessCollection:
        return self._process_manager.all()

    def logs(self, path: str) -> LogFile:
        return self._log_manager.open(path)

    def workflow(self, name: str) -> WorkflowBuilder:
        return self._workflow_manager.create(name)

    def run(self, name: str):
        return self._workflow_manager.run(name)
```

processes/ Module

process.py — The Process Object

```
class Process:
    def __init__(self, pid, name, memory_mb, cpu_percent):
        self.pid = pid
        self.name = name
        self.memory_mb = memory_mb
        self.cpu_percent = cpu_percent

    def kill(self):
        # os.kill(self.pid, signal.SIGKILL)
        pass

    def terminate(self):
```

```

        # os.kill(self.pid, signal.SIGTERM)
        pass

    def details(self) -> dict:
        return {
            'pid': self.pid,
            'name': self.name,
            'memory_mb': self.memory_mb,
            'cpu_percent': self.cpu_percent
        }

    def __repr__(self):
        return f'Process({self.name}, pid={self.pid},
mem={self.memory_mb:.1f}MB)'

```

factory.py — ProcessFactory

```

import psutil

class ProcessFactory:
    @staticmethod
    def create(proc) -> Process:
        try:
            with proc.oneshot():
                return Process(
                    pid=proc.pid,
                    name=proc.name(),
                    memory_mb=proc.memory_info().rss / 1024 / 1024,
                    cpu_percent=proc.cpu_percent()
                )
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            return None

```

manager.py — ProcessManager

```

import psutil

class ProcessManager:
    def all(self) -> ProcessCollection:
        processes = []
        for proc in psutil.process_iter():
            p = ProcessFactory.create(proc)
            if p:
                processes.append(p)
        return ProcessCollection(processes)

```

```

class ProcessCollection:
    def __init__(self, data: list):
        self.data = data

    def named(self, name: str):
        self.data = [p for p in self.data if name.lower() in
p.name.lower()]
        return self

    def memory_above(self, mb: float):
        self.data = [p for p in self.data if p.memory_mb > mb]
        return self

    def cpu_above(self, percent: float):
        self.data = [p for p in self.data if p.cpu_percent > percent]
        return self

    def sort_by_memory(self):
        self.data = sorted(self.data, key=lambda p: p.memory_mb,
reverse=True)
        return self

    def sort_by_cpu(self):
        self.data = sorted(self.data, key=lambda p: p.cpu_percent,
reverse=True)
        return self

    def all(self) -> list:
        return self.data

    def summarize(self) -> str:
        lines = [f'{p.name}: {p.memory_mb:.1f} MB' for p in
self.data[:10]]
        return '\n'.join(lines)

    def __iter__(self):
        return iter(self.data)

```

logs/ Module

logfile.py — LogFile Object

```

class LogFile:
    def __init__(self, path: str):
        self.path = path

```

```

        with open(path, 'r') as f:
            self._lines = f.readlines()
            self._filtered = self._lines[:]

    def errors(self):
        self._filtered = [l for l in self._filtered if 'ERROR' in l]
        return self

    def warnings(self):
        self._filtered = [l for l in self._filtered if 'WARNING' in l]
        return self

    def contains(self, keyword: str):
        self._filtered = [l for l in self._filtered if keyword in l]
        return self

    def tail(self, n: int):
        self._filtered = self._filtered[-n:]
        return self

    def all(self) -> list:
        return self._filtered

    def summarize(self) -> str:
        total    = len(self._lines)
        errors   = sum(1 for l in self._lines if 'ERROR' in l)
        warns    = sum(1 for l in self._lines if 'WARNING' in l)
        return f'Total: {total} lines | Errors: {errors} | Warnings: {warns}'

```

manager.py — LogManager

```

class LogManager:
    def open(self, path: str) -> LogFile:
        return LogFile(path)

```

workflows/ Module

This is the most complex and impressive part of your work. Take time to get it right.

WorkflowStep — Base Class

```

from abc import ABC, abstractmethod

```

```

class WorkflowStep(ABC):
    @abstractmethod
    def execute(self, context: dict) -> dict:
        pass

    @abstractmethod
    def to_dict(self) -> dict:
        pass

    @classmethod
    @abstractmethod
    def from_dict(cls, data: dict):
        pass

```

Concrete Step Examples

```

class ProcessFilterStep(WorkflowStep):
    def __init__(self, memory_above=None, cpu_above=None, named=None):
        self.memory_above = memory_above
        self.cpu_above = cpu_above
        self.named = named

    def execute(self, context):
        procs = context.get('processes', server.processes())
        if self.memory_above:
            procs = procs.memory_above(self.memory_above)
        if self.cpu_above:
            procs = procs.cpu_above(self.cpu_above)
        if self.named:
            procs = procs.named(self.named)
        context['processes'] = procs
        return context

    def to_dict(self):
        return { 'type': 'process_filter',
                  'memory_above': self.memory_above,
                  'cpu_above': self.cpu_above,
                  'named': self.named }

```

StepFactory

```

class StepFactory:
    _registry = {
        'process_filter': ProcessFilterStep,
        'sort': SortStep,
        'log_filter': LogFilterStep,
    }

```

```

        'tail':          TailStep,
        'summary':       SummaryStep,
        'export':        ExportStep,
    }

    @classmethod
    def create(cls, data: dict) -> WorkflowStep:
        step_type = data['type']
        klass = cls._registry.get(step_type)
        if not klass:
            raise ValueError(f'Unknown step type: {step_type}')
        return klass.from_dict(data)

```

Workflow Object

```

import json, os
from datetime import datetime

WORKFLOW_DIR = os.path.expanduser('~/.serverkit/workflows/')

class Workflow:
    def __init__(self, name: str):
        self.name = name
        self.steps = []
        self.created_at = datetime.utcnow().isoformat()
        self.last_run = None

    def add_step(self, step: WorkflowStep):
        self.steps.append(step)
        return self

    def run(self):
        context = {}
        for step in self.steps:
            print(f' Running: {step.__class__.__name__}')
            context = step.execute(context)
        self.last_run = datetime.utcnow().isoformat()
        return context

    def save(self):
        os.makedirs(WORKFLOW_DIR, exist_ok=True)
        path = os.path.join(WORKFLOW_DIR, f'{self.name}.json')
        with open(path, 'w') as f:
            json.dump(self.to_dict(), f, indent=2)
        print(f'Workflow saved: {path}')

    def to_dict(self) -> dict:
        return {

```



```

        'name':      self.name,
        'created_at': self.created_at,
        'last_run':  self.last_run,
        'steps':     [s.to_dict() for s in self.steps]
    }

    @classmethod
    def from_dict(cls, data: dict):
        wf = cls(data['name'])
        wf.created_at = data.get('created_at')
        wf.last_run   = data.get('last_run')
        wf.steps = [StepFactory.create(s) for s in data['steps']]
        return wf

    def export(self, path: str):
        with open(path, 'w') as f:
            json.dump(self.to_dict(), f, indent=2)

```

WorkflowBuilder — Builder Pattern

```

class WorkflowBuilder:
    def __init__(self, name: str):
        self._workflow = Workflow(name)
        self._current_resource = None

    def processes(self):
        self._current_resource = 'processes'
        return self

    def memory_above(self, mb: float):
        self._workflow.add_step(ProcessFilterStep(memory_above=mb))
        return self

    def sort_by_memory(self):
        self._workflow.add_step(SortStep(field='memory'))
        return self

    def summarize(self):
        self._workflow.add_step(SummaryStep())
        return self

    def save(self):
        self._workflow.save()
        return self._workflow

    def build(self) -> Workflow:
        return self._workflow

```

WorkflowManager

```
class WorkflowManager:
    def create(self, name: str) -> WorkflowBuilder:
        return WorkflowBuilder(name)

    def run(self, name: str):
        path = os.path.join(WORKFLOW_DIR, f'{name}.json')
        with open(path) as f:
            data = json.load(f)
            wf = Workflow.from_dict(data)
            print(f'Running workflow: {wf.name}')
            return wf.run()

    def list(self) -> list:
        if not os.path.exists(WORKFLOW_DIR):
            return []
        return [f.replace('.json', '') for f in os.listdir(WORKFLOW_DIR)]

    def import_workflow(self, path: str):
        with open(path) as f:
            data = json.load(f)
            wf = Workflow.from_dict(data)
            wf.save()
            return wf
```

OOP Patterns You Demonstrate

Pattern	Where You Use It
Encapsulation	Process hides psutil internals. LogFile hides file I/O. Managers hide system calls.
Composition	Server holds managers. Workflow holds Steps. Collection holds Process objects.
Inheritance + ABC	WorkflowStep is abstract. All step types implement execute(), to_dict(), from_dict().
Factory Pattern	ProcessFactory.create(). StepFactory.create() with type registry.
Builder Pattern	WorkflowBuilder chains .processes().memory_above(1000).sort_by_memory().save()
Fluent Interface	Every filter method returns self so calls chain naturally.
Composite Pattern	Workflow is composed of Step objects, each independently executable.

Shared Contracts with Dev 2

Dev 2 (Shell + AI) depends on these. Do not change signatures without coordinating.

Method / Interface	Contract
<code>server.processes()</code>	Returns ProcessCollection
<code>server.logs(path)</code>	Returns LogFile
<code>server.workflow(name)</code>	Returns WorkflowBuilder
<code>server.run(name)</code>	Loads and runs saved workflow by name
<code>.all()</code>	Returns list of objects (terminal)
<code>.summarize()</code>	Returns str (terminal)
<code>Workflow.to_dict()</code>	Returns the canonical JSON schema
<code>Workflow.from_dict(data)</code>	Reconstructs from JSON (used by AI layer)

Suggested Build Order

Build in this order so you can test each layer before adding the next:

Milestone	Acceptance Test
1. Process + ProcessCollection	Core data model. Test: <code>.memory_above()</code> <code>.sort_by_memory()</code> <code>.all()</code>
2. ProcessFactory + ProcessManager	psutil integration. Test: real process list from OS
3. Server class (processes only)	Entry point works. Test: <code>server.processes().all()</code>
4. LogFile + LogManager	Test: <code>.errors()</code> <code>.contains()</code> <code>.tail()</code> on a sample log file
5. WorkflowStep + subclasses	Test: each step's <code>execute()</code> and <code>to_dict()</code> independently
6. Workflow + serialization	Test: <code>save()</code> and <code>from_dict(to_dict())</code> round-trip
7. WorkflowBuilder	Test: full chain saves correct JSON
8. WorkflowManager.run()	Test: load from disk and execute

Tip: Write one test per milestone before moving on. You do not need a full test suite — just enough to know each layer works before building on top of it.